



Java

*Lombok;
Associação entre
classes; Json*

PAULO R T CÂNDIDO

Download

https://projectlombok.org/download

Project Lombok

Features▼

Community▼

Order / Donate

Install▼

Download

 [Download 1.18.46](#)

Explorer

▼ POOJava4



▼ lib

 lombok.jar

Metodos getters, setters, toString e hashCode gerados automaticamente

▼ POOJava4
 > lib
 ▼ pacote1
 J Main.java

> Outline
> Timeline

pacote1 > J Main.java > Main > main(String[])
2
3 import lombok.*;
4
5 @Data // gera getters, setters, toString, equals e hashCode
6 class Aluno{
7 private final int id;
8 private final String nome;
9 }
10
11 public class Main {
 Run | Debug
12 public static void main(String[] args){
13 Aluno aluno = new Aluno(id: 10, nome: "João");
14 System.out.println(aluno.getNome() + " - " + aluno.toString());
15
16 // Testando o equals
17 // Dois objetos com o mesmo id e nome devem ser considerados iguais
18 Aluno aluno2 = new Aluno(id: 10, nome: "João");
19 System.out.println(aluno.equals(aluno2));
20 }
21 }

Define quais métodos e comportamento

pacote1 > J Main.java > ...

```
1  package pacote1;
2
3  import lombok.*;
4
5  // Especificar detalhadaente quais métodos e quais campos devem ser incluídos no toString e equals/hashCode
6  @Getter // método get para todos os campos
7  @Setter // metodo set apenas para os campos que não são finais
8  @RequiredArgsConstructor // construtor com parâmetros para todos os campos final e @NonNull
9  @ToString(onlyExplicitlyIncluded = true) // apenas os campos anotados com @ToString.Include serão incluídos no toString
10 @EqualsAndHashCode(onlyExplicitlyIncluded = true) // apenas os campos anotados com @EqualsAndHashCode.Include serão incluídos no equals e hashCode
11 class Aluno{
12     @ToString.Include
13     @EqualsAndHashCode.Include // unico campo incluído no equals e hashCode - dois objetos com o mesmo id serão considerados iguais
14     private final int id; // apenas atributos final e @NonNull são incluídos no construtor gerado pelo @RequiredArgsConstructor
15
16     @ToString.Include
17     @NonNull // apenas atributos final e @NonNull são incluídos no construtor gerado pelo @RequiredArgsConstructor
18     private String nome;
19
20     private int idade; // não incluído no construtor, nem no toString e equals/hashCode
21 }
22
23 public class Main {
24     Run | Debug
25     public static void main(String[] args){
26         Aluno aluno = new Aluno(id: 10, nome: "João"); // não possui idade no construtor, pois não é final nem @NonNull
27         aluno.setIdade(idade: 15);
28         System.out.println(aluno.getNome() + " - " + aluno.toString());
29     }
30 }
```

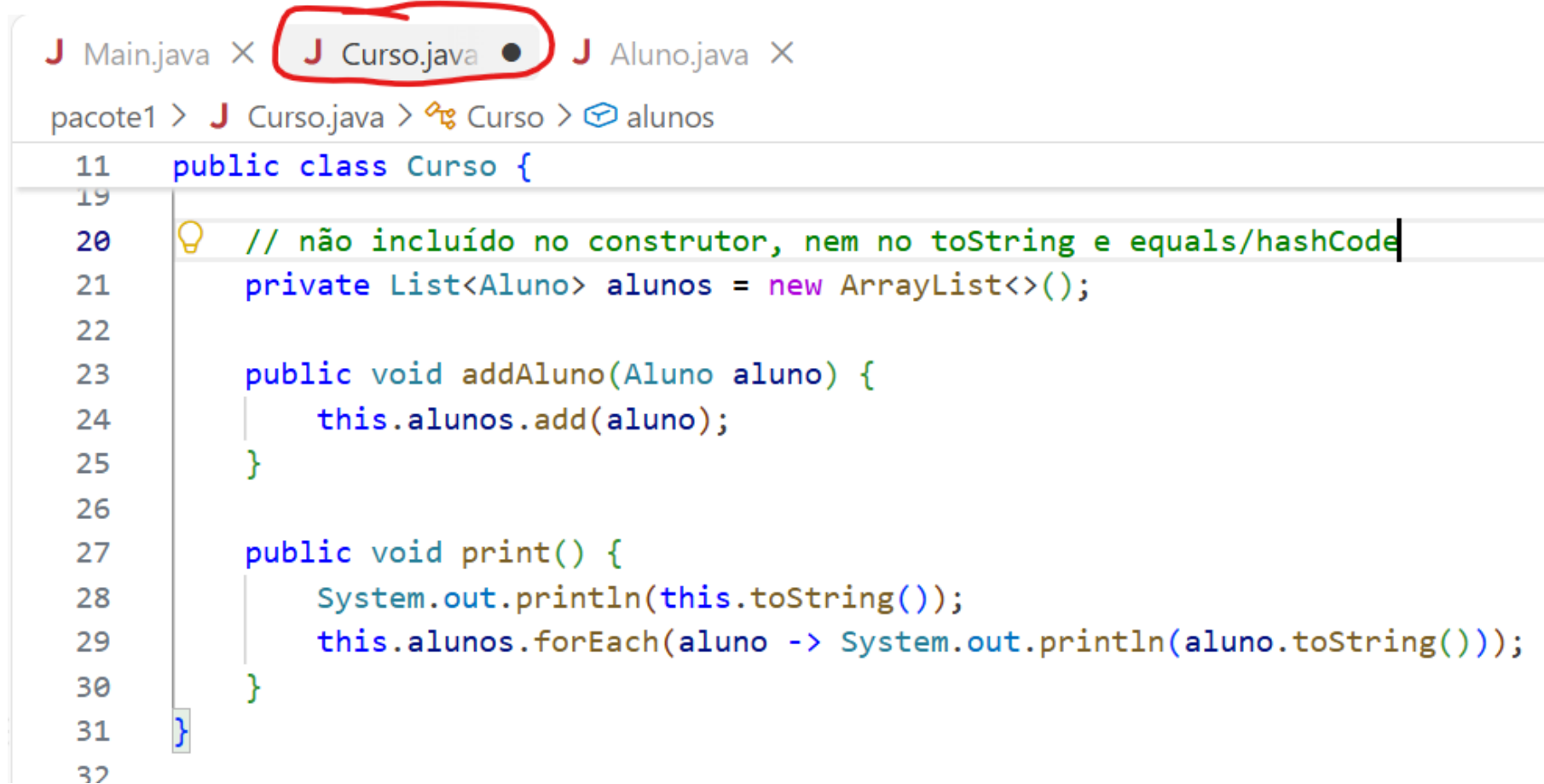
Exercício

- 1) Mover classe Aluno para um arquivo próprio: Aluno.java
- 2) Criar classe Curso (atributos: id e nome). Usando Lombok, configurar métodos getter, toString e EqualsHashCode (considerar apenas o id). Arquivo: Curso.java

```
1 package pacote1;
2
3 import java.util.List;
4 import lombok.*;
5
6 @Getter // método get para todos os atributos
7 @RequiredArgsConstructor // construtor com argumentos obrigatórios
8 @ToString(onlyExplicitlyIncluded = true)
9 @EqualsAndHashCode(onlyExplicitlyIncluded = true)
10 public class Curso {
11     @EqualsAndHashCode.Include
12     @ToString.Include
13     private final int id;
```

Alunos do Curso

Adicionar à classe Curso propriedade correspondendo a relação de alunos associados ao curso. Adicionar à Curso os métodos “addAluno(adiciona aluno a lista de alunos) e “print” que imprime no terminal dados do curso e de todos os alunos associados a ele.



```
11 public class Curso {
19
20 // não incluído no construtor, nem no toString e equals/hashCode
21 private List<Aluno> alunos = new ArrayList<>();
22
23 public void addAluno(Aluno aluno) {
24     this.alunos.add(aluno);
25 }
26
27 public void print() {
28     System.out.println(this.toString());
29     this.alunos.forEach(aluno -> System.out.println(aluno.toString()));
30 }
31 }
32
```

Testar

```
3 public class Main {
    Run | Debug
4     public static void main(String[] args){
5         Aluno aluno = new Aluno(id: 10, nome: "João"); // não possi
6         aluno.setIdade(idade: 15);
7         System.out.println(aluno.getNome() + " - " + aluno.toStrir
8
9         // Testando o equals
10        // Dois objetos com o mesmo id e nome devem ser considera
11        Aluno aluno2 = new Aluno(id: 10, nome: "Maria");
12        System.out.println(aluno.equals(aluno2));
13
14        Curso curso = new Curso(id: 1, nome: "ADS");
15        curso.addAluno(aluno);
16        curso.addAluno(aluno2);
17        curso.print();
18    }
19 }
```

Instalar Jackson no projeto – Gerador de JSON

1. Baixar o jackson-databind (Principal):jackson-databind.

Acesse: mvnrepository.com/artifact/com.fasterxml.jackson.core/jackson-databind

Clique na versão mais recente e estável (ex: 2.17.0 ou superior).

Na tabela da página, procure pela linha **Files** e clique no link **jar** para baixar o arquivo.

2. Baixar o jackson-core:jackson-core.

Acesse: mvnrepository.com/artifact/com.fasterxml.jackson.core/jackson-core

Selecione **a mesma versão** do passo anterior (ex: 2.17.0).

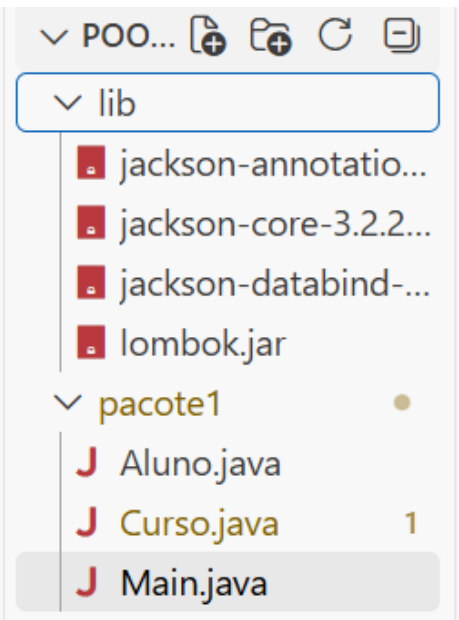
Clique em **jar** na linha *Files* para baixar.

3. Baixar o jackson-annotations:jackson-annotations.

Acesse: mvnrepository.com/artifact/com.fasterxml.jackson.core/jackson-annotations

Selecione **a mesma versão** usada nos anteriores.

Clique em **jar** na linha *Files* para baixar.



Incluir Jackson – renomear propriedades

```
J Main.java X J Curso.java ● J Aluno.java X
pacote1 > J Curso.java > Curso > id
5 import com.fasterxml.jackson.databind.ObjectMapper;
6 import tools.jackson.databind.ObjectMapper;
7 import com.fasterxml.jackson.annotation.JsonProperty;
8 //import com.fasterxml.jackson.annotation.JsonIgnore;
9
10 @Getter // método get para todos os campos
11 @RequiredArgsConstructor //construtor com parâmetros para todos os car
12 @ToString(onlyExplicitlyIncluded = true) // apenas os campos anotados
13 @EqualsAndHashCode(onlyExplicitlyIncluded = true) // apenas os campos
14 public class Curso {
15     @EqualsAndHashCode.Include // unico campo incluído no equals e has
16     @ToString.Include
17     ⚡ @JsonProperty("codigoCurso")
18     private final int id;
19
20     @NonNull
21     @ToString.Include
22     @JsonProperty("nomeCurso")
23     private String nome;
24
25     // não incluído no construtor, nem no toString e equals/hashCode
26     @JsonProperty("listaAlunos")
27     private List<Aluno> alunos = new ArrayList<>();
```

Incluir Jackson – método toJson()

J Main.java X J Curso.java X J Aluno.java X

pacote1 > J Curso.java > Curso

```
15 public class Curso {
33
34     public void print() {
35         System.out.println(this.toString());
36         this.alunos.forEach(aluno -> System.out.println(aluno.toString()));
37     }
38
39     public String toJson(){
40         return new ObjectMapper().writeValueAsString(this);
41     }
42 }
43
```

J Main.java X J Curso.java X J Aluno.java X

pacote1 > J Main.java > Main > main(String[])

```
3 public class Main {
4     public static void main(String[] args){
12         System.out.println(aluno.equals(aluno2));
13
14         Curso curso = new Curso(id: 1, nome: "ADS");
15         curso.addAluno(aluno);
16         curso.addAluno(aluno2);
17         curso.print();
18         System.out.println(curso.toJson());
19     }
20 }
21
```